

L12 · Απληστοί Αλγόριθμοι II

Ελάχιστη Καθυστέρηση & Τοπολογική Διάταξη

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Χρονοπρογραμματισμός με ελάχιστη μέγιστη καθυστέρηση (Earliest Deadline First), η τεχνική του επιχειρήματος ανταλλαγής με αντιστροφές, οι τρεις τεχνικές απόδειξης απληστων αλγορίθμων, και η τοπολογική διάταξη σε DAG.

1 Τι θα δούμε

Δύο θέματα — και μία τεχνική απόδειξης που θα ξαναδείς σε **κάθε** εξεταστική:

1. **Χρονοπρογραμματισμός με ελάχιστη καθυστέρηση** — και το **επιχείρημα ανταλλαγής** (exchange argument), η δεύτερη μεγάλη τεχνική για απληστους αλγορίθμους μετά το «greedy stays ahead» του προηγούμενου μαθήματος.
2. **Τοπολογική διάταξη** — πώς βάζεις σε σειρά εργασίες με προαπαιτούμενα, και γιατί αυτό είναι δυνατό **αν και μόνο αν** δεν υπάρχουν κύκλοι.

2 Χρονοπρογραμματισμός με Ελάχιστη Καθυστέρηση

2.1 Διατύπωση

Μία μηχανή, που επεξεργάζεται **μία εργασία τη φορά**.

- Η εργασία j απαιτεί t_j μονάδες χρόνου και έχει **προθεσμία** d_j .
- Αν ξεκινήσει τη στιγμή s_j , τελειώνει τη στιγμή $f_j = s_j + t_j$.
- **Καθυστέρηση** της j : $\ell_j = \max\{0, f_j - d_j\}$ — πόσο **μετά** την προθεσμία της τελειώσει (ή 0 αν πρόλαβε).

Στόχος: προγραμμάτισε όλες τις εργασίες ελαχιστοποιώντας τη **μέγιστη** καθυστέρηση $L = \max_j \ell_j$.

Διαίσθηση

Πρόσεξε τη διαφορά από το προηγούμενο μάθημα. Εκεί διαλέγαμε ένα υποσύνολο· εδώ προγραμματίζουμε **όλες** τις εργασίες — απλώς αποφασίζουμε τη **σειρά**. Και δεν μετράμε άθροισμα καθυστερήσεων αλλά τη **χειρότερη** καθυστέρηση: μία πολύ αργοπορημένη εργασία χαλάει τη λύση, ακόμη κι αν όλες οι άλλες είναι στην ώρα τους.

2.2 Υποψήφια κριτήρια

Πάλι το ίδιο απληστο πρότυπο: βάλε τις εργασίες σε **κάποια σειρά**.

Κριτήριο	Σειρά	Αποτέλεσμα
Μικρότερος χρόνος επεξεργασίας	αύξουσα t_j	αποτυγχάνει
Μικρότερο χρονικό περιθώριο	αύξουσα $d_j - t_j$	αποτυγχάνει
Μικρότερη προθεσμία	αύξουσα d_j	δουλεύει ✓

Προσοχή

Γιατί αποτυγχάνει το «μικρότερος χρόνος επεξεργασίας»: αγνοεί τελείως την προθεσμία — μια σύντομη εργασία με προθεσμία στο μέλλον προηγείται μιας μακράς εργασίας που λήγει αμέσως.
Γιατί αποτυγχάνει το «μικρότερο περιθώριο»: μια εργασία με μικρό περιθώριο αλλά μακρινή προθεσμία μπορεί να καθυστερήσει μία άλλη με κοντινή προθεσμία.

2.3 Ο αλγόριθμος: Earliest Deadline First

Κλειδί

Άπληστος αλγόριθμος. Ταξινόμησε τις εργασίες σε **αύξουσα σειρά προθεσμίας** ($d_1 \leq d_2 \leq \dots \leq d_n$) και προγραμματίσέ τις **τη μία μετά την άλλη**, χωρίς κενά.

ΕλάχιστηΚαθυστερήση(t, d):

```

Ταξινόμησε τις εργασίες ώστε  $d_1 \leq d_2 \leq \dots \leq d_n$ 
χρόνος  $\leftarrow 0$ 
Για  $j = 1$  έως  $n$ :
    ανάθεσε στην εργασία  $j$  το διάστημα  $[\text{χρόνος}, \text{χρόνος} + t_j]$ 
     $s_j \leftarrow \text{χρόνος}$  ;  $f_j \leftarrow \text{χρόνος} + t_j$ 
     $\text{χρόνος} \leftarrow \text{χρόνος} + t_j$ 
επίστρεψε τα διαστήματα  $[s_j, f_j]$ 

```

Διαίσθηση

Διαισθητικά: «κάνε πρώτα ό,τι λήγει πρώτο». Είναι ακριβώς αυτό που κάνεις πριν την εξεταστική περίοδο — το μάθημα με την κοντινότερη ημερομηνία πάει πρώτο, ασχέτως πόσο διάβασμα θέλει.

2.4 Δύο παρατηρήσεις-κλειδιά

Κλειδί

Αδρανής χρόνος (idle time): στιγμές που η μηχανή δεν δουλεύει.

- Υπάρχει **πάντα** μια βέλτιστη λύση **χωρίς** αδρανή χρόνο (αν η μηχανή κάθετα, σπρώξε τα πάντα νωρίτερα — καμία καθυστέρηση δεν μεγαλώνει).
- Ο **άπληστος** αλγόριθμος, εξ ορισμού, δεν έχει αδρανή χρόνο.

2.5 Αντιστροφές

Κλειδί

Αντιστροφή (inversion): ένα ζεύγος εργασιών i, j με $d_i < d_j$ — δηλαδή η i λήγει νωρίτερα — αλλά η j προγραμματίζεται **πριν** την i .

Ο άπληστος αλγόριθμος, που ταξινομεί ακριβώς κατά προθεσμία, **δεν έχει καμία αντιστροφή**. Αυτό είναι το «δαχτυλικό αποτύπωμα» της λύσης του.

Διαίσθηση

Κρίσιμη παρατήρηση: αν ένα χρονοδιάγραμμα χωρίς αδρανή χρόνο έχει έστω **μία** αντιστροφή, τότε έχει και μία αντιστροφή με τις δύο εργασίες **διαδοχικά** προγραμματισμένες. (Σκέψου το πιο «κοντινό» αντεστραμμένο ζεύγος — δεν μπορεί να έχει τίποτα ανάμεσά τους εκτός σειράς.)

2.6 Το επιχείρημα ανταλλαγής

Κλειδί

Ισχυρισμός. Η αντιμετάθεση δύο **διαδοχικών, αντεστραμμένων** εργασιών μειώνει τον αριθμό των αντιστροφών κατά 1 και **δεν αυξάνει** τη μέγιστη καθυστέρηση.

Απόδειξη. Έστω i, j διαδοχικές με αντιστροφή ($d_i < d_j$, αλλά η j πριν την i). Αντιμεταθέτουμε τις. Έστω ℓ_k οι παλιές και ℓ'_k οι νέες καθυστερήσεις.

- Για κάθε $k \neq i, j$: η εργασία k δεν μετακινήθηκε καθόλου, άρα $\ell'_k = \ell_k$.
- Η i μετακινήθηκε **νωρίτερα**, άρα τελειώνει νωρίτερα: $\ell'_i \leq \ell_i$.
- Η j μετακινήθηκε αργότερα. Τελειώνει τώρα εκεί όπου τελείωνε **πριν** η i — δηλαδή $f'_j = f_i$. Άρα:

$$\ell'_j = f'_j - d_j = f_i - d_j \leq f_i - d_i = \ell_i$$

(η ανισότητα ισχύει αφού $d_j > d_i$).

Άρα και οι δύο νέες καθυστερήσεις, ℓ'_i και ℓ'_j , είναι $\leq \ell_i \leq$ της παλιάς μέγιστης. Η μέγιστη καθυστέρηση **δεν αυξήθηκε**. ■

Κλειδί

Θεώρημα. Η λύση S του άπληστου αλγορίθμου είναι **βέλτιστη**.

Απόδειξη (εις άτοπον). Έστω S^* μια βέλτιστη λύση **χωρίς αδρανή χρόνο** και με τον **ελάχιστο δυνατό αριθμό αντιστροφών**.

- Αν το S^* δεν έχει αντιστροφές: τότε είναι ταξινομημένο κατά προθεσμία, άρα $S = S^*$ — ο άπληστος είναι βέλτιστος. ✓
- Αν το S^* έχει τουλάχιστον μία αντιστροφή: τότε έχει και μία **διαδοχική** αντιστροφή. Αντιμεταθέτοντάς την, παίρνουμε λύση με **αυστηρά λιγότερες** αντιστροφές που **δεν αυξάνει** τη μέγιστη καθυστέρηση — δηλαδή εξίσου βέλτιστη. Αυτό **αντιφάσκει** με την επιλογή του S^* ως βέλτιστης με τις ελάχιστες αντιστροφές. ■

2.7 Οι τρεις τεχνικές απόδειξης άπληστων αλγορίθμων

Κλειδί

1. **Ο άπληστος προηγείται** (greedy stays ahead) — δείχνεις ότι μετά από κάθε βήμα η μερική λύση του άπληστου είναι τουλάχιστον τόσο καλή όσο κάθε άλλης. (Προηγούμενο μάθημα, χρονοπρογραμματισμός διαστημάτων.)
2. **Επιχείρημα ανταλλαγής** (exchange argument) — μετατρέπεις σταδιακά μια οποιαδήποτε βέλτιστη λύση σε αυτήν του άπληστου, χωρίς να χαλάει η βελτιστότητα. (Αυτή η διάλεξη.)
3. **Δομική εξήγηση** (structural bound) — αποδεικνύεις ένα κάτω/άνω φράγμα μέσα στο πρόβλημα και δείχνεις ότι ο αλγόριθμος το πετυχαίνει. (Προηγούμενο μάθημα, διαμέριση διαστημάτων — το βάθος.)

3 Τοπολογική Διάταξη

3.1 Το πρόβλημα

Πολλές εργασίες με **προαπαιτούμενα**: η εργασία u πρέπει να γίνει **πριν** την v . Τα μοντελοποιούμε με ένα **κατευθυνόμενο γράφημα** $G = (V, A)$: η ακμή (u, v) σημαίνει «η u προηγείται της v ».

Κλειδί

Τοπολογική διάταξη του G : μια διάταξη των κορυφών $v_1, v_2, \dots, v_n$ τέτοια ώστε **κάθε** ακμή (v_i, v_j) να πηγαίνει «προς τα εμπρός» — δηλαδή $i < j$.

Διαίσθηση

Διαισθητικά: βάζεις όλες τις εργασίες σε μία γραμμή έτσι ώστε κάθε βέλος να δείχνει **δεξιά**. Αν τα καταφέρεις, η σειρά αυτή είναι ένα έγκυρο πρόγραμμα εκτέλεσης: κάθε εργασία γίνεται αφού έχουν τελειώσει όλα τα προαπαιτούμενά της.

Εφαρμογές: γράφημα προαπαιτούμενων μαθημάτων, σειρά μεταγλώττισης αρχείων κώδικα, διοχέτευση (pipeline) υπολογιστικών εργασιών, υπολογισμός παράστασης όπως $((a + b) \cdot (c + d) + e) \cdot (a + b)$.

3.2 Πότε υπάρχει;

Δεν έχουν **όλα** τα κατευθυνόμενα γραφήματα τοπολογική διάταξη. Αν $a \rightarrow b \rightarrow c \rightarrow a$, ποια μπαίνει πρώτη; Καμία — υπάρχει **κύκλος**.

Κλειδί

Θεώρημα. Ένα κατευθυνόμενο γράφημα έχει τοπολογική διάταξη **αν και μόνο αν** είναι **άκυκλο** (Directed Acyclic Graph — DAG).

Μέρος 1 — **αν έχει τοπολογική διάταξη, τότε είναι DAG**. Έστω ότι το G έχει τοπολογική διάταξη $v_1, \dots, v_n$ και έναν κύκλο C . Έστω v_i η κορυφή του C με τον **μικρότερο** δείκτη, και v_j η κορυφή ακριβώς **πριν** την v_i μέσα στον C . Τότε η (v_j, v_i) είναι ακμή. Από την επιλογή της v_i ως μικρότερης, $i < j$. Αλλά αφού (v_j, v_i) είναι ακμή σε τοπολογική διάταξη, $j < i$. **Άτοπο. ■**

Μέρος 2 — κάθε DAG έχει κορυφή χωρίς εισερχόμενες ακμές. Έστω ότι **κάθε** κορυφή έχει εισερχόμενη ακμή. Ξεκίνα από μια κορυφή v και ακολούθησε ακμές **προς τα πίσω**: αφού η v έχει εισερχόμενη (u, v) , πήγαινε στην u : αφού η u έχει εισερχόμενη, συνέχισε. Με n κορυφές, μετά από n βήματα θα συναντήσεις κάποια κορυφή **δύο φορές** — και το κομμάτι ανάμεσα στις δύο επισκέψεις είναι **κύκλος**. Άτοπο, αφού το G είναι DAG. ■

Μέρος 3 — κάθε DAG έχει τοπολογική διάταξη. Επαγωγή στο πλήθος κορυφών n .

- **Βάση:** $n = 1$ — προφανές.
- **Επαγωγικό βήμα:** έστω DAG με $k + 1$ κορυφές. Από το Μέρος 2, υπάρχει κορυφή v **χωρίς εισερχόμενες** ακμές. Το $G \setminus \{v\}$ είναι DAG με k κορυφές, άρα (επαγωγική υπόθεση) έχει τοπολογική διάταξη. Τοποθετούμε την v **πρώτη**: αφού δεν έχει εισερχόμενες ακμές, κάθε ακμή της δείχνει προς τα εμπρός. Η διάταξη είναι έγκυρη. ■

3.3 Ο αλγόριθμος

Η απόδειξη του Μέρους 3 **είναι** ο αλγόριθμος: βρίσκεις επανειλημμένα μια κορυφή χωρίς εισερχόμενες ακμές και την «βγάζεις».

```

ΤοπολογικήΔιάταξη(G):
  βρες κορυφή v του G χωρίς εισερχόμενες ακμές
  L <- ΤοπολογικήΔιάταξη(G \ {v})
  επίστρεψε v, L           // η v μπροστά, μετά η υπόλοιπη λίστα

```

Κλειδί

Πολυπλοκότητα: $O(m + n)$. Με αφελή υλοποίηση θα ήταν $O(n^2)$ (ψάχνεις κάθε φορά από την αρχή). Το κόλπο: διατήρησε

- $\text{count}[w]$ = πλήθος εναπομεινάντων **εισερχόμενων** ακμών της w .
- S = σύνολο (λίστα) κορυφών με $\text{count} = 0$.

Αρχικοποίηση: μία σάρωση του γραφήματος, $O(m + n)$. **Διαγραφή της v :** βγάλε την από το S ($O(1)$): για κάθε ακμή (v, w) μείωσε το $\text{count}[w]$, και αν φτάσει 0 βάλε την w στο S — $O(1)$ ανά ακμή. Συνολικά $O(m + n)$.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **Ελάχιστη μέγιστη καθυστέρηση** — μία μηχανή, εργασίες με (t_j, d_j) , καθυστέρηση $\ell_j = \max\{0, f_j - d_j\}$. Σωστό κριτήριο: **Earliest Deadline First** (αύξουσα προθεσμία).
2. **Αντιστροφή** = ζεύγος εργασιών εκτός σειράς προθεσμίας. Ο άπληστος δεν έχει καμία.
3. **Επιχείρημα ανταλλαγής** — αντιμετάθεση μιας διαδοχικής αντιστροφής μειώνει τις αντιστροφές κατά 1 χωρίς να μεγαλώνει το L . Άρα κάθε βέλτιστη λύση μπορεί να γίνει η λύση του άπληστου.
4. **Τρεις τεχνικές απόδειξης:** greedy stays ahead, exchange argument, structural bound.
5. **Τοπολογική διάταξη** — διάταξη κορυφών ώστε κάθε ακμή να δείχνει εμπρός. Υπάρχει **αν και μόνο αν** το γράφημα είναι **DAG** (άκυκλο).
6. Αλγόριθμος: βγάξε επανειλημμένα κορυφή χωρίς εισερχόμενες ακμές — $O(m + n)$ με $\text{count}[]$ και σύνολο πηγών.